Repository design document for the GitHub repository "recuperated_Brayton_HTHP" for the paper "Techno-economic assessment of thermal energy storage integrated reverse Brayton cycles for provision of high-temperature process heat"

Samuele Grandin[1,*], Andrea Vecchi[1] and Adriano Sciacovelli[1,*]

[1] Department of Civil and Mechanical Engineering, Technical University of Denmark (DTU), Kongens Lyngby 2800 (DK)

[*] Corresponding authors: samgr@dtu.dk; adrsc@dtu.dk

1. Overview

The recuperated_Brayton_HTHP repository contains the code, input data, and results behind the paper above. It studies the integration of sensible thermal energy storage (TES) into recuperated Brayton high-temperature heat pumps (HTHPs). The thermodynamic plants are modelled in TESPy. A mixed-integer optimization (Pyomo with GLPK or HiGHS) schedules each plant against day-ahead electricity prices, and the resulting operation is turned into a levelized cost of heat (LCOH).
Four plant layouts are assessed. They differ in whether TES is integrated and in whether the compression is intercooled. In every layout the cycle is recuperated.

Label	File-name prefix	Description
S-BR	`S_BR_hthp`	Standalone (no TES), base recuperated cycle
S-IR	`S_IR_hthp`	Standalone (no TES), intercooled recuperated cycle
TES-BR	`TES_BR_hthp`	TES-integrated, base recuperated cycle
TES-IR	`TES_IR_hthp`	TES-integrated, intercooled recuperated cycle

The assessment, for each layout and for the whole comparison, is assessed in the following way:

1. Plant definition
2. Design simulation
3. Validation
4. Performance, exergy and sizing results
5. Cost estimation
6. Operational optimization on 2025 prices (DK1 and DE, fixed and free heat demand)
7. LCOH with price forecasts
8. Comparison across layouts

Section 5 describes each step.
The intended outcome is that a reader can clone the repository, set up the environment as in Section 3, run the workflow and regenerate the results of the paper (Section 5).

2. Design Principles

The repository is designed according to the following main objectives:

- Modularity: separate the physical model, input data, validation, and post-processing functions into dedicated modules. Their orchestration is done afterwards in a main simulation file.
- Reproducibility: define the model configuration, the simulation process-flow, and required Python dependencies explicitly so that the repository can be cloned and the case studies can be reproduced in a dedicated environment.
- Traceability: maintain a clear relationship between input data, model configuration, functions, and simulation results.

- Reusability: allow the existing model structure and processing functions to be reused for future configurations or extensions.
- Separation of concerns: distinguish between the definition of the HTHP and TES model, its execution, and the processing and visualization of the resulting data.

3. Computational environment initialization

3.1 Requirements

- Python 3.12.2. The code was developed on Windows with Visual Studio Code.
- The Python packages in `requirements.txt`: `tespy`, `numpy`, `matplotlib`, `pandas`, `CoolProp`, `fluprodia`, `pyomo`, highspy.
- The GLPK solver (`glpsol`) is installed on the system and reachable from the PATH. It is not a Python package, so pip install does not provide it. The operational optimization calls it through Pyomo (`solver_name="glpk"`). If the solver is not installed, the optimization adopts automatically `"HiGHS"` which is included in Python library.

3.2 Installation

Clone the repository and create a virtual environment in its root folder (PowerShell on Windows):

```
git clone [GitHub URL]
cd recuperated_Brayton_HTHP
python -m venv tespy_env
.\tespy_env\Scripts\Activate.ps1
python -m pip install -r requirements.txt
```

Optional: install GLPK and check that it is found

```
glpsol –version
```

3.3 Running in Visual Studio Code

The scripts import from the `src` package (`from src import ...`) and write to paths relative to the repository root (for example `results/plots`). Two settings are therefore needed:

1. Open the repository root as the VS Code workspace folder**.** The scripts must run with the repository root as working directory.
2. Select the virtual environment as interpreter. Press Ctrl+Shift+P, choose “Python: Select Interpreter”, and pick `tespy_env` (`.\tespy_env\Scripts\python.exe`).

4. Folder Architecture

The repository is organized into dedicated folders according to the different functions required to develop, execute, validate, and document the HTHP case study. The structure separates input data, source code, execution scripts, generated results, documentation, examples, and testing.
The overall repository structure is:

Commented [SG1]: Check from external user

Commented [SG2]: Check if the steps presented are enough

```
recuperated_Brayton_HTHP/
├── data/        raw/ and processed/ inputs (validation data, electricity prices)
├── docs/        documentation and references
├── examples/    short runnable examples
├── main/        entry-point scripts (workflow)
├── results/     plots/, tables/
├── src/         core code: geometry, input, models, post_processing + helper
modules
├── tests/       automated tests (empty)
├── requirements.txt
└── README.md
```

The separation is intended to improve maintainability, reproducibility, and traceability of the calculations.

4.1 Data/

Input datasets, kept apart from the code. `raw/` holds original files and is not modified. `processed/` holds the cleaned files read by the code.

File	Content and use
`processed/Benvenuti_validation_recuperated.csv`	Reference state points of the recuperated Brayton heat pump of Benvenuti et al. [1].
`raw/electricity_prices/Day-ahead_prices_202501010000_202601010000_Hour.csv`	Hourly day-ahead prices for 2025 for several bidding zones. Source: [2].
`processed/electricity_prices/dk1_electricity_prices_2025.csv` `processed/electricity_prices/de_electricity_prices_2025.csv`	Hourly DK1 and DE prices 2025.
`processed/electricity_prices/dk1_energinet_2025_2050_scenarios.csv` `processed/electricity_prices/de_vbw_2025_2050_scenarios.csv`	Yearly price scenarios (low, central, high) 2025 to 2050 from Energinet and VBW, with factors relative to 2025. Intermediate years are linearly interpolated. Source: [3], [4].

4.2 Docs/

Documentation of the repository containing the repository design document and its references.

4.3 Examples/

Short scripts showing typical uses of the code. They run the same way as the `main/` scripts.

Commented [SG3]: To be extended

The `layouts/` folder contains single HTHP runnable scripts, delivering the same thermodynamic and operational analysis as for the whole techno-economic assessment.
The four HTHPs are then evaluated starting from the single layouts results in `hthp_comparison.py`, generating same results as the paper's ones.
Finally, `operational_strategies_comparison.py` presents some examples of how the operational strategies are executed for different layouts with different heat demand schedules.

4.4 Main/

The full runnable techno-economic assessment workflow in the paper design conditions, generating the paper results: `run_techno_economic_assessment.`
The sub-modular working is explained in Section 5.

4.5 Results/

All paths below are relative to `results/`. File names start with the layout prefix of Section 1. Figures that compare layouts start with `all_hthp_`. Plots and tables are saved respectively as PNG and CSV files.

Location	Content
`paper/`	Tables and plots adopted in the paper.
`paper/plots/`	T-s and heat exchanger diagrams per layout.
`paper/tables/`	Component cost per layout. Component exergy destruction and efficiency. Thermodynamic state of every connection. Heat-exchanger tables. LCOH tables by scenario, market and demand type, and the discounted cost breakdown of the central scenario. Performance indicators per layout. Component sizing per layout.

4.6 Src/

All core functionalities imported in the `main/`.

Module or folder	Content
`input/`	Plant definition. `components/`, `connections/` and `parameters/` each hold one JSON file per layout plus the function that applies it (`set_plant_components`, `set_plant_connections`, `set_plant_parameters`). `styling/` holds the common plot style (PLOT_STYLE) and the helper function for figure generation `create_figure.py`.
`network_creator.py`	`create_configured_network()` returns an empty TESPy network with the default units (°C, bar, kJ/kg, MW, kJ/kgK).
`models/hthp_models.py`	One class per layout, all built on `hthp_model`. Each class points to its three JSON files, and `build_into(network)` assembles components,

Commented [SG4]: Eventually we save as figures just what we use in the paper, what is not in the paper is either not saved or a csv files

TO BE DISCUSSED

Commented [SG5]: Eventually, to change and correct.

Module or folder	Content
	connections and parameters. `standalone_base_recup_hthp_Benvenuti` reuses the S-BR topology with the parameters of the validation case.
`geometry/tanks.py`	`calculate_tanks_geometry`: sizes the TES tanks (volume, mass, height, diameter, energy) for HITEC molten salt from the solved plant and a storage duration.
`post_processing/`	Plotting and tables: `ts_diagram.py`, `hx_diagram.py`, `parameters_tables.py` (performance, sizing and connection tables), `plant_validation.py` (`validate_plant`), `exergy_plotting.py`, `cost_plotting.py` (cost, LCOH and ΔLCOH figures), `economic_plotting.py` (operational strategy).
`exergy_analysis.py`	`get_exergy_analysis`: component exergy destruction and exergetic efficiencies.
`cost_analysis.py`	`calculate_component_cost`: component and tank cost estimate.
`plant_operation.py`	`optimize_operational_strategy`: MILP of HTHP and TES operation against hourly prices.
`lcoh_analysis.py`	`calculate_future_costs` and `perform_lcoh_analysis`.
`techno_economic_analysis.py`	Perform the techno-economic analysis for the paper

4.7 Tests/

Space for reproducible validation and regression tests (currently empty).

5. Techno-economic assessment

5.1 Entry point

The techno-economic assessment runs through the command `run_techno_economic_assessment` which gets as inputs the layouts to simulate, the price zones where to operate, the typologies of heat demand schedules to be provided, and which layouts to be compared on one or more parameters. The save path is provided as well.
Eventually, the full year simulations are bypassed setting the input `lcoh_results=True`, which adopts the existing LCOH results for the post-processing stage.
The general runnable function is the following:

```
run_techno_economic_assessment(
    layouts=["S-BR", "S-IR", "TES-BR", "TES-IR"],
    price_zones=["DK1", "DE"],
    heat_demand_schedule=["fixed", "variable"],
    comparison={
        "ts_diagram": ["S-BR", "S-IR"],
        "ts_diagram": [["S-BR", "TES-BR"], ["S-IR", "TES-IR"]],
        "hx_diagram": ["S-BR", "S-IR"],
```

```
        ("exergy_destruction", "exergy_efficiency"): ["S-BR", "S-IR", "TES-BR",
"TES-IR"],
        "capex_breakdown": ["S-BR", "S-IR", "TES-BR", "TES-IR"],
        "operational_strategy": [["S-BR", "TES-BR"], "DK1", ["fixed", "variable"],
"week 1"],
        "lcoh_scenario": [["S-BR", "S-IR", "TES-BR", "TES-IR"], "DK1", "fixed"],
        "delta_lcoh": [[["S-BR", "TES-BR"], ["S-IR", "TES-IR"]], ["DK1", "DE"],
                       ["fixed", "variable"], ["low", "central", "high"]],
        "lcoh_breakdown": [["S-BR", "TES-BR", "S-IR", "TES-IR"], ["DK1", "DE"],
"fixed", "central"],
    },
    save_path="results/paper",
)
```

`layouts`, `price_zones` and `heat_demand_schedule` define the cases that are simulated: with the values above, four plants, each optimized on two price zones and two demand schedules (16 yearly runs). Each entry of `comparison` then selects a subset of those cases and one figure.

The `comparison` input is explained in the following table:

Key	Arguments	Figure produced
`ts_diagram`	`[layouts]` or `[[layouts], [layouts], ...]`	One T-s diagram for each of the listed cycle layouts, or comparative T-s diagrams for sets of listed cycle layouts.
`hx_diagram`	`[[layouts], [layouts], ...]`	One hx diagram for each of the listed cycle layouts.
`("exergy_destruction", "exergy_efficiency")`	`[layouts]`	One figure with two panels: (a) exergy destruction per component as one stacked bar per layout, (b) exergetic efficiency per component per layout.
`capex_breakdown`	`[layouts]`	One stacked bar per layout, split by component.
`operational_strategy`	`[layouts, zone, demand schedules, "week n"]`. Here `[["S-BR", "TES-BR"], "DK1", ["fixed", "variable"], "week 1"]`.	A grid of panels: one column per layout, one row per heat demand schedule. Representation done in a determined time and price zone.
`lcoh_scenario`	`[layouts, zone, demand schedule]`. Here `[[four layouts], "DK1", "fixed"]`.	Grouped bars of the LCOH [€/MWh]: one group per scenario (low, central, high), one bar per layout, for the chosen zone and heat demand schedule.
`delta_lcoh`	`[pairs, zones, demand schedules, scenarios]`. Each pair is `[standalone, TES-integrated]`.	ΔLCOH = LCOH(TES-integrated) − LCOH(standalone). One panel per price zone, scenarios on the x axis, marker shape for the demand schedule, colour for the layout pair.
`lcoh_breakdown`	`[layouts, zones, demand schedule, scenario]`. Here `[[four layouts], ["DK1", "DE"], "fixed", "central"]`.	Stacked bars of the LCOH contributions per layout. One panel per price zone, one bar per layout. Heat demand and price scenario are fixed.

Each figure is saved in save_path (results/paper) as PNG. The plot functions name their files <file_name>_<figure>.png (for example _ts_diagram, _exergy_combined, _cost_stacked, _lcoh_scenarios, _delta_lcoh, _lcoh_breakdown, _operational_strategy).

5.2 Workflow steps

The following are the workflow steps for the entire simulation. First all the layouts are solved, and the relevant data are generated, then the required comparison takes place.

#	Step	Code

1	Create the HTHP case	`layout=create_configured_network`
		`layout.build_into(network)`
		`layout.solve(mode="design")`

2	Validation (S-BR)	`validate_plant` with `data/processed/Benvenuti_validation_recuperated.csv`

3	Parameter tables	`generate_performance_parameters_table`
		`generate_sizing_parameters_table`
		`generate_connections_table`
		`save_hx_table`

4	T-s and heat-exchanger diagrams	`plot_ts_diagram` `plot_hx_diagram`
5	Exergy analysis	`get_exergy_analysis`
6	TES tank sizing (TES layouts)	`calculate_tanks_geometry`
7	Component cost	`calculate_component_cost`
8	Operation over 2025, DK1 and DE, fixed and free demand	`optimize_operational_strategy`
9	LCOH for each case	`perform_lcoh_analysis`
10	Repeat for all layouts	

11	Coparison plots and figures	`plot_ts_diagram`
		`plot_hx_diagram`
		`plot_exergy_plots`
		`plot_component_cost_stacked`
		`plot_operational_strategy`
		`plot_lcoh_scenarios_stacked`
		`plot_delta_lcoh_analysis`
		`plot_lcoh_breakdown_stacked`

Commented [SG6]: Consider renaming it

References

[1] M. Benvenuti, G. F. Frate, and L. Ferrari, “Evaluating Brayton Heat Pump Potential for Industrial Decarbonisation,” *J. Phys. Conf. Ser.*, vol. 2893, no. 1, p. 012117, Nov. 2024, doi: 10.1088/1742-6596/2893/1/012117.

[2] “SMARD | SMARD - Market data.” Accessed: Oct. 22, 2026. [Online]. Available: https://www.smard.de/en

[3] “Energinet, ‘Baggrundsnotat om elprisfremskrivninger i Analyseforudsætninger til Energinet 2025 (AF25).’”
[4] C. Krüger, L. Springorum, F. Merten, and F. Auschra, “Die Auswirkung unterschiedlicher Treiberentwicklungen auf den Strompreis”.